Title: Smart Outcome Predictor
Duration: 6 Hours

🎯 Objective

The objective of this project is to design a **robust ensemble-based machine learning system** capable of predicting outcomes accurately by combining **multiple weak and strong learners**. Students will explore **bagging, boosting, voting, and stacking strategies**, and understand how ensemble methods improve **generalization, stability, and performance** in both **classification and regression tasks**.

📑 Problem Statement

You are working as a **Machine Learning Engineer** at an EdTech analytics company. The platform wants to build a **Smart Outcome Predictor** that can:

- **Classification Task:** Predict whether a student will *successfully complete* an online course.
- **Regression Task:** Predict the *final performance score* of a student.

Single models are producing unstable results. Your manager asks you to design an **ensemble-based solution** that improves accuracy and robustness by combining multiple models.

🧠 Part A: Conceptual Foundation (Theory)

Answer briefly (2–4 lines each):

1. What are **Ensemble Learning Methods** and why are they effective?
2. Explain the difference between **Bagging, Boosting, and Voting**.

3. What is **Bias–Variance Trade-off**, and how do ensemble methods address it?
4. Explain **Voting Classifier** and **Stacking Ensemble**.
5. Compare **AdaBoost, Gradient Boosting, LightGBM, and XGBoost** conceptually.

🧪 Part B: Dataset Understanding & Preparation

You are provided with a dataset containing:

- Student demographic indicators
- Engagement metrics (time spent, sessions, quiz attempts)
- Assessment scores
- Target variables:
  - `completion_status` → Classification (0 = Not Completed, 1 = Completed)
  - `final_score` → Regression (0–100)

Dataset: Access from here

Tasks:

6. Identify features and target variables for both tasks.
7. Perform a **train–test split** for classification and regression problems.
8. Apply basic preprocessing (scaling/encoding if required).

📦 Part C: Bagging (Bootstrap Aggregating)

9. Implement **Bagging Classifier** for course completion prediction.
10. Implement **Bagging Regressor** for final score prediction.
11. Compare bagging results with a single base model.

⚡ Part D: Boosting Algorithms

AdaBoost

12. Implement **AdaBoost Classifier**.
13. Implement **AdaBoost Regressor**.

14. Analyze how weak learners improve sequentially.

Gradient Boosting

15. Implement **Gradient Boosting Classifier**.
16. Implement **Gradient Boosting Regressor**.
17. Analyze learning rate and number of estimators.

Light Gradient Boosting

18. Implement **LightGBM Classifier**.
19. Implement **LightGBM Regressor**.
20. Analyze performance and training efficiency.

XGBoost

21. Implement **XGBoost Classifier**.
22. Implement **XGBoost Regressor**.
23. Compare performance and robustness with other boosting models.

🗳️ Part E: Voting & Stacking Ensembles

24. Implement **Voting Classifier** using multiple base classifiers.
25. Compare **Hard Voting vs Soft Voting**.
26. Implement **Stacking Classifier** with a meta-learner.
27. Implement **Stacking Regressor** for score prediction.

📊 Part F: Model Evaluation & Comparison

28. Evaluate **classification models** using:
    - Accuracy
    - Precision, Recall, F1-Score
    - ROC-AUC
29. Evaluate **regression models** using:
    - MAE, RMSE
    - R^2 Score
30. Compare all ensemble techniques and identify the **best performing model** for each task.

📄 Part G: Final Analysis & Reporting

31. Prepare a final report covering:
    - Impact of bagging vs boosting
    - Comparison of tree-based boosting algorithms
    - Advantages of voting and stacking
    - Final model recommendation for deployment
32. Submit:
    - Source code / Jupyter Notebook
    - Evaluation tables and plots
    - Final conclusions and business interpretation

✅ Submission Guidelines

- Include practical implementation in a Jupyter Notebook and screenshots.
- Label all charts clearly and write short interpretations under each result.
- **GitHub Repository:**
    - Create a GitHub repository to host your project.
    - Upload your project files, including source code, and documentation to the repository.
    - Add a document (PDF) explaining theory concepts with definitions (if any).
    - Ensure that you provide a clear and descriptive README.md file.

Remember to follow the instructions provided professionally, make suitable assumptions wherever necessary, and avoid copying code or content from unauthorized sources. Good luck with your project work!

Smart Outcome Predictor

Supervised Learning
